ATTENDEASE · SOFTWARE ENGINEERING MINI PROJECT

System Validation Report

Document ID	AE-SVR-01
Version	1.0
Date	10 November 2026
Build under test	`main` @ `92bcd23` (merge of PR #113, 1 Nov 2026)
Prepared by	Arundhati K (PES1UG24CS083)
Team	Arundhati K, Anshdeep Singh Sachdeva, Atharva Ashish Vyas, Bhoomika Dayanand Jituri

End-to-end validation of the AttendEase system against Test Plan cases TC-01 to TC-13.

1. Summary

13/13
TEST CASES PASSED

43
API REQUESTS EXECUTED

79
CHECKS (ALL PASSED)

0.23s
P95 AT 50 CONCURRENT USERS

System validation checks the assembled application against the 13 representative test cases of the Test Plan (§13), end to end: real HTTP requests go to a running server, a real database and real JWT authentication, as a client would use them. Each case's *expected result* is checked literally. Where the expected result describes a server-side effect, it is also checked on the server: the password hash, the email sent, audit fields, the security log entry and the database contents.

Result: all **13 of 13** test cases passed: 71 API assertions and 8 server-side checks, with no failures.

TC	Test case	Requirements	Method	Result
TC-01	Register a new student account	FR-001, SEC-001	End-to-end API + server checks	PASS
TC-02	Lockout after repeated failed logins	SEC-002	End-to-end API + server checks	PASS
TC-03	Admin assigns faculty; faculty sees the section	FR-013	End-to-end API + server checks	PASS
TC-04	Start session and bulk-mark all present	FR-014, FR-016, FR-017	End-to-end API + server checks	PASS
TC-05	Duplicate attendance record rejected (409)	FR-037	End-to-end API + server checks	PASS
TC-06	Student attendance dashboard	FR-020, FR-021	End-to-end API + server checks	PASS
TC-07	Student raises a correction; faculty emailed	FR-024, FR-025	End-to-end API + server checks	PASS
TC-08	Faculty approves a correction; audit fields set	FR-027, SEC-006	End-to-end API + server checks	PASS
TC-09	Student calling an admin-only endpoint gets 403	SEC-003	End-to-end API + server checks	PASS
TC-10	Defaulter report for a section	FR-029, FR-031	End-to-end API + server checks	PASS
TC-11	50 concurrent users; 80-student roster	NFR-001, NFR-002	Load test + automated tests	PASS

TC-12	Service-layer coverage ≥ 70%	NFR-006	coverage.py	PASS
TC-13	Student cannot access another student's record	SEC-004	End-to-end API + server checks	PASS

2. Test environment and data

Item	Detail
Server	Gunicorn (1 sync worker), as in `Dockerfile` / `docker-compose.yml`, on 127.0.0.1
Database	SQLite, fresh database with all migrations applied (PostgreSQL in the Docker deployment)
Email	Django console email backend: outgoing mail is written to the server log and inspected
API client	Newman (Postman CLI): collection `AttendEase_SystemValidation.postman_collection.json`, 11 folders, one per case
Load generator	Python with 50 concurrent threads (TC-11)
Test data	1 admin; 3 faculty (plus 1 created in TC-03); 8 demo students and 80 load-test students; courses CS341 and CS342 (4 weeks of closed sessions, 2 pending corrections) and CS399 (80 students, for TC-11). Rohan Das and Sneha Kulkarni are below the 75% threshold by design. This exceeds the Test Plan §6 minimum of 1 admin, 2 faculty and 10 students across 2 courses.
Executed	10 November 2026, 06:37 UTC, build `92bcd23`

Cases run in order on one database, as a user session would: TC-04 creates the session that TC-05 uses, and TC-06 sees it after it is closed.

3. Test case results

TC-01 · Register a new student account

PASS

Requirement(s)	FR-001, SEC-001
Description	Register a new student account with valid details.
Expected result	Account created; confirmation shown; password stored hashed.
Actual result	201 Created with the new account's id and role; the response contains no password. The new account logs in (200). The stored password is a PBKDF2-SHA256 hash with 600,000 iterations.

#	Step	HTTP	ms	Checks
1	Register a student with valid details `POST /api/auth/register`	201	248	✔ HTTP 201 ✔ Account id returned (confirmation) ✔ Role is student ✔ Password is never echoed back ✔ No password field in response
2	New account can log in `POST /api/auth/login`	200	91	✔ HTTP 200
✓	Password stored as a salted PBKDF2 hash, not plain text (DB)		pass	`pbkdf2_sha256$600000$…`

TC-02 · Lockout after repeated failed logins

PASS

Requirement(s)	SEC-002
Description	Attempt login with an incorrect password repeatedly on one account, exceeding the configured failure threshold.
Expected result	Account is locked/throttled once the threshold is reached; further attempts are rejected until the cooldown, even with the correct password.
Actual result	Attempts 1–5 with a wrong password: 401. Attempt 6 with the correct password: 429 and no token. Attempt 7: 429 again. A different account logs in normally (200).

#	Step	HTTP	ms	Checks
1	Wrong password, attempt 1 `POST /api/auth/login`	401	89	✔ HTTP 401
2	Wrong password, attempt 2 `POST /api/auth/login`	401	87	✔ HTTP 401
3	Wrong password, attempt 3 `POST /api/auth/login`	401	108	✔ HTTP 401
4	Wrong password, attempt 4 `POST /api/auth/login`	401	92	✔ HTTP 401
5	Wrong password, attempt 5 `POST /api/auth/login`	401	84	✔ HTTP 401
6	Correct password after threshold is still refused `POST /api/auth/login`	429	4	✔ HTTP 429 ✔ No token issued while locked
7	Further attempt during cooldown is refused `POST /api/auth/login`	429	5	✔ HTTP 429
8	Other accounts are unaffected `POST /api/auth/login`	200	82	✔ HTTP 200
✓	Failure counter persisted for the locked account (DB)		pass	`1 login-attempt row(s)`

TC-03 · Admin assigns faculty; faculty sees the section

PASS

Requirement(s)	FR-013
Description	Admin assigns a faculty member to a course-section.
Expected result	Assignment saved; faculty sees the section on their dashboard.
Actual result	Course CS360 and section A created (201), faculty assigned (200). The faculty member's `my-sections` lists exactly that section.

#	Step	HTTP	ms	Checks
1	Admin logs in `POST /api/auth/login`	200	83	✔ HTTP 200
2	Register a faculty member `POST /api/auth/register`	201	85	✔ HTTP 201
3	Admin creates course CS360 `POST /api/academic/courses`	201	10	✔ HTTP 201
4	Admin creates section A `POST /api/academic/courses/4/sections`	201	12	✔ HTTP 201
5	Admin assigns the faculty member to the section `POST /api/academic/assignments`	200	8	✔ HTTP 200
6	Faculty logs in `POST /api/auth/login`	200	87	✔ HTTP 200
7	Faculty sees the section on their dashboard `GET /api/academic/my-sections`	200	8	✔ HTTP 200 ✔ CS360 section A is listed ✔ Only the assigned section is listed

TC-04 · Start session and bulk-mark all present

PASS

Requirement(s)	FR-014, FR-016, FR-017
Description	Faculty starts an attendance session and bulk-marks all students present with a single action.
Expected result	Session and attendance records created; roster reflects marked status for every student after the single bulk action.
Actual result	Session started for today's scheduled CS341 period (201, open). One mark-all-present call (200). The roster shows all 6 enrolled students as present.

#	Step	HTTP	ms	Checks
1	Faculty logs in `POST /api/auth/login`	200	81	✔ HTTP 200
2	Find today's scheduled CS341 period `GET /api/academic/my-sections`	200	6	✔ HTTP 200 ✔ Scheduled period found
3	Faculty starts the attendance session `POST /api/attendance/sessions/start`	201	9	✔ HTTP 201 ✔ Session is open
4	Bulk-mark all students present in a single action `POST /api/attendance/sessions/22/mark-all-present`	200	11	✔ HTTP 200
5	Roster reflects present for every student `GET /api/attendance/sessions/22/roster`	200	5	✔ HTTP 200 ✔ All 6 enrolled students listed ✔ Every student is present

TC-05 · Duplicate attendance record rejected (409)

PASS

Requirement(s)	FR-037
Description	Faculty attempts to mark attendance twice for the same session/student.
Expected result	Second attempt rejected with a duplicate-record error (409).
Actual result	A second record for the same student and session returned 409 Conflict. The roster still shows exactly one entry for that student, still present (no overwrite).

#	Step	HTTP	ms	Checks
1	Second attendance record for the same student and session `POST /api/attendance/sessions/22/records`	409	6	✔ HTTP 409
2	Original mark is not overwritten `GET /api/attendance/sessions/22/roster`	200	6	✔ HTTP 200 ✔ Still exactly one entry for the student ✔ Still present
3	Faculty closes the session `POST /api/attendance/sessions/22/close`	200	9	✔ HTTP 200

TC-06 · Student attendance dashboard

PASS

Requirement(s)	FR-020, FR-021
Description	Student views their attendance dashboard.
Expected result	Correct attendance percentage and per-course breakdown displayed.
Actual result	The dashboard lists both enrolled courses. Each percentage equals an independent recount from the student's session history, and each below-threshold flag is correct.

#	Step	HTTP	ms	Checks
1	Student logs in `POST /api/auth/login`	200	82	✔ HTTP 200
2	Full session history (for an independent recount) `GET /api/attendance/dashboard/history?from=2026-01-01&to=2026-11-10`	200	8	✔ HTTP 200
3	Student views the attendance dashboard `GET /api/attendance/dashboard`	200	9	✔ HTTP 200 ✔ Per-course breakdown for both enrolled courses ✔ CS341 percentage = 7/13 = 53.85 ✔ CS341 below-threshold flag is correct ✔ CS342 percentage = 4/8 = 50 ✔ CS342 below-threshold flag is correct

TC-07 · Student raises a correction; faculty emailed

PASS

Requirement(s)	FR-024, FR-025
Description	Student raises a correction request for a disputed record.
Expected result	Request created with Pending status; faculty notified by email.
Actual result	201 Created with status PENDING, and the request appears in the assigned faculty member's queue. One notification email was sent, only to the assigned faculty member.

#	Step	HTTP	ms	Checks
1	Student logs in `POST /api/auth/login`	200	83	✔ HTTP 200
2	Student raises a correction for a disputed record `POST /api/corrections`	201	15	✔ HTTP 201 ✔ Request is PENDING
3	Assigned faculty sees it in the pending queue `GET /api/corrections`	200	6	✔ HTTP 200 ✔ Listed as PENDING
✓	Request stored with PENDING status (DB)	pass		`[('PENDING',)]`
✓	Notification email sent to the assigned faculty only (server mail log)	pass		`Subject: AttendEase: attendance correction request #3`

TC-08 · Faculty approves a correction; audit fields set

PASS

Requirement(s)	FR-027, SEC-006
Description	Faculty approves a pending correction request.
Expected result	Attendance record updated; request marked Approved; audit fields (acted_by, acted_at) populated on the CorrectionRequest.
Actual result	200 OK, status APPROVED. `acted_by` is the approving faculty member's id and `acted_at` is a server timestamp. The underlying record changed from absent to present.

#	Step	HTTP	ms	Checks
1	Find Ananya's pending correction `GET /api/corrections`	200	6	✔ HTTP 200
2	Faculty identity `GET /api/auth/me`	200	3	✔ HTTP 200
3	Faculty approves the pending request `PATCH /api/corrections/1`	200	8	✔ HTTP 200 ✔ Request marked APPROVED ✔ acted_by is the approving faculty ✔ acted_at is populated
4	Attendance record is updated to present `GET /api/attendance/sessions/20/roster`	200	5	✔ HTTP 200 ✔ Ananya is now present
✓	acted_by / acted_at populated on CorrectionRequest (DB)	pass		`status=APPROVED, acted_by_id=2, acted_at=2026-11-10 06:37:13.831116`

TC-09 · Student calling an admin-only endpoint gets 403

PASS

Requirement(s)	SEC-003
Description	Student account attempts to call an admin-only academic-setup endpoint.
Expected result	403 Forbidden returned; action logged.
Actual result	Both admin-only calls (create course, assign faculty) returned 403. The server logged a warning for each, naming the user, role, view and method.

#	Step	HTTP	ms	Checks
1	Student calls admin-only create-course endpoint `POST /api/academic/courses`	403	3	✔ HTTP 403
2	Student calls admin-only faculty-assignment endpoint `POST /api/academic/assignments`	403	3	✔ HTTP 403
✓	Each denied call logged with user, role, view and method (server log)	pass		`Role access denied: user_id=7 role=student view=CourseListView method=POST | Role access denied: user_id=7 role=student view=AssignmentView method=POST`

TC-10 · Defaulter report for a section

PASS

Requirement(s)	FR-029, FR-031
Description	Generate a defaulter report for a section below the shortage threshold.
Expected result	Report lists exactly the students below threshold with correct percentages.
Actual result	The report lists exactly Rohan Das and Sneha Kulkarni. Their percentages match an independent SQL recount of all 6 enrolled students.

#	Step	HTTP	ms	Checks
1	Defaulter report for CS341 section A `GET /api/reports/defaulters?section_id=1`	200	14	✔ HTTP 200 ✔ Lists exactly the students below threshold ✔ Every listed percentage is below its threshold
✓	Report equals an independent SQL recount of every enrolled student	pass		`recount: Ananya Sharma 100.0%, Karthik Iyer 84.62%, Priya Nair 100.0%, Rahul Verma 100.0%, Rohan Das 53.85%, Sneha Kulkarni 69.23%; threshold 75.0%`

TC-11 · 50 concurrent users; 80-student roster

PASS

Requirement(s)	NFR-001, NFR-002
Description	Simulate 50 concurrent faculty/student requests while loading the attendance-marking roster for an 80-student section, and measure response time.
Expected result	Roster loads within 3 seconds; 95% of API requests complete within 2 seconds under the 50-concurrent-user load.
Actual result	Under 50 concurrent users, the 95th-percentile response time was **0.23 s** for the 80-student roster and **0.24 s** for mixed student requests. Every request finished within 2 s, with no errors. A single roster load takes 6 ms.

Load test against the running server

Server: Gunicorn with 1 sync worker, the same command as the project's `Dockerfile` and `docker-compose.yml`, on 2 vCPUs, with the load generator on the same machine. Section CS399 has 80 enrolled students and an open session. 50 worker threads send requests at the same time: 500 roster requests in phase 1, and in phase 2 500 student requests from 50 separately logged-in students (dashboard, history, my-sections, me).

Measure	Roster, 50 concurrent	Mixed API, 50 students	Target
Requests / errors	500 / 0	500 / 0	no errors
Median response	0.168 s	0.176 s	
95th percentile	**0.229 s**	**0.241 s**	≤ 2 s (NFR-001)
Slowest request	0.234 s	0.267 s	
Requests within 2 s	100.0%	100.0%	≥ 95%
Throughput	261.7 req/s	280.0 req/s	
Single roster load (80 students)	**0.006 s** (HTTP 200, 80 students)		≤ 3 s (NFR-002)

Automated performance tests (run on every CI build)

Test	Time (s)	Result
`ConcurrentLoadTests.test_nfr001_mixed_api_requests_under_concurrent_load`	0.48	Passed
`ConcurrentLoadTests.test_tc11_50_concurrent_roster_requests_within_2_seconds`	0.72	Passed
`RosterPerformanceTests.test_nfr002_marking_actions_for_80_students_stay_within_3_seconds`	0.11	Passed
`RosterPerformanceTests.test_nfr002_roster_for_80_students_loads_within_3_seconds`	0.09	Passed
`RosterPerformanceTests.test_nfr002_roster_query_count_does_not_grow_with_class_size`	0.09	Passed

> **Notes.** (1) The production stack uses PostgreSQL; this run used SQLite with the same application code (deviation D-09). (2) For comparison, the same load against Django's single-process development server (`runserver`, not used in deployment) gave a p95 of 1.51 s for the roster and 2.09 s for mixed requests (94.6% within 2 s). So the 2-second target depends on running a production WSGI server, as the Docker setup does.

TC-12 · Service-layer coverage ≥ 70%

PASS

Requirement(s)	NFR-006
Description	Run the service-layer automated test suite and measure statement/branch coverage.
Expected result	Automated test coverage on service-layer business logic is at least 70%.
Actual result	Service-layer statement coverage is **93.9%** and branch coverage is **92.3%** (320 tests, all passing). CI enforces the 70% minimum on every pull request. Details are in AE-CCR-01 and AE-BCR-01.

Service-layer measure	Covered	Total	Coverage	Target
Statements	542	577	**93.9%**	≥ 70%
Branches	155	168	**92.3%**	≥ 70%

TC-13 · Student cannot access another student's record

PASS

Requirement(s)	SEC-004
Description	A student authenticated as Student A attempts to view or act on Student B's attendance record via a direct API call with Student B's record ID.
Expected result	403 Forbidden (or empty/filtered result) returned; Student A cannot read or modify a record outside their own enrolled sections.
Actual result	Correction on another student's record: 404, with nothing disclosed. Reading that record's audit history: 404. Opening the section roster: 403. The student's own history contains only her own rows. No request was created in the database.

#	Step	HTTP	ms	Checks
1	Student A (Ananya) logs in `POST /api/auth/login`	200	81	✔ HTTP 200
2	Student A raises a correction on Student B's record `POST /api/corrections`	404	5	✔ Refused with 403/404 ✔ Nothing about the record is disclosed
3	Student A reads Student B's audit history `GET /api/attendance/sessions/20/records/7/history`	404	4	✔ Refused with 403/404
4	Student A opens the section roster (all students' marks) `GET /api/attendance/sessions/20/roster`	403	3	✔ HTTP 403
5	Student A's own history contains only her own rows `GET /api/attendance/dashboard/history?from=2026-01-01&to=2026-11-10`	200	7	✔ HTTP 200 ✔ No other student's data in the payload
✓	No correction request created on Student B's record (DB)	pass		`0 request(s) on record 111`

4. Requirements validated

Through these cases the following requirements were validated end to end: FR-001, FR-003, FR-004, FR-013, FR-014, FR-015, FR-016, FR-017, FR-018, FR-020, FR-021, FR-022, FR-024, FR-025, FR-026, FR-027, FR-029, FR-031, FR-037, NFR-001, NFR-002, NFR-006, SEC-001, SEC-002, SEC-003, SEC-004 and SEC-006. All other functional requirements are covered by the automated suite (Unit Test Report AE-UTR-01, section 5).

5. Open items

- **SEC-005 (HTTPS):** not validated here. The settings contain no HTTPS enforcement (`SECURE_SSL_REDIRECT`, HSTS, secure cookies), so TLS must be terminated by the hosting web server or proxy. Recommended before a public deployment.
- **NFR-001 on PostgreSQL:** the load test used SQLite. Repeating TC-11 against the `docker-compose` stack would confirm the result on the production database (see D-09).
- **TIME_ZONE = UTC:** "today" for sessions follows UTC, so between 00:00 and 05:30 IST a session for the Indian date counts as a future date (D-07). This does not affect class hours.

6. Conclusion

All 13 test cases of the Test Plan pass on the final build, and no defects were found during system validation. The system meets its functional, security and performance acceptance criteria for submission. The open items above concern deployment configuration, not application behaviour.